SIMATS ENGINEERING
ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA15

COURSE OUTCOME COVERED
CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct
I Dinesh.S(192521152) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Assessment Tasks
Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

1. Cloud Architecture for a Video Streaming Platform

A video streaming platform must support millions of users simultaneously while providing continuous streaming with minimum buffering.

Scalability and Caching

- Use **horizontal scaling** to add more cloud servers when the number of users increases.
- Use a **Content Delivery Network (CDN)** to store frequently accessed video content at edge locations close to users.
- CDN caching reduces latency and decreases the load on the main application servers.
- Store original videos in scalable **cloud object storage**.
- Use adaptive bitrate streaming to automatically adjust video quality according to network conditions.
- Cache popular and frequently requested video segments to improve streaming performance.

Load Balancing

- A **load balancer** distributes incoming user requests among multiple streaming servers.
- It prevents a single server from becoming overloaded.
- Health checks can identify failed servers and stop sending requests to them.
- Global load balancing can direct users to the nearest or healthiest region.
- This improves availability and provides uninterrupted streaming.

Auto-Scaling

- Auto-scaling automatically increases the number of servers when demand increases.
- It can use metrics such as CPU usage, network traffic, request rate, and number of active users.
- During low-demand periods, unnecessary servers are removed.
- This ensures that sufficient resources are available during peak traffic.

Cost Optimization

- Use CDN caching to reduce bandwidth consumption from the origin servers.
- Use **pay-as-you-go** resources for unpredictable workloads.
- Use reserved or committed cloud resources for predictable workloads.
- Move rarely accessed videos to cheaper storage tiers.
- Automatically scale down unused resources.
- Monitor storage, bandwidth, and computing costs regularly.

Conclusion

Using **CDN caching, load balancing, auto-scaling, scalable storage, and cost management techniques** allows the video streaming platform to handle millions of users efficiently with minimum buffering and controlled cloud expenses.

2. Containerized Fintech Application Using Multi-Cloud

A fintech startup can use container orchestration and a multi-cloud strategy to maintain consistent application deployment across different cloud providers.

Container Orchestration

- Use **Kubernetes** as the container orchestration platform.
- Package applications and their dependencies into containers.
- Use the same container images across different cloud providers.
- Kubernetes automatically manages container deployment, scheduling, restarting, and scaling.
- Use Kubernetes manifests or Helm charts to maintain consistent configurations.

Deployment Management

- Use a **CI/CD pipeline** to automate testing and deployment.
- Use Infrastructure as Code to create consistent cloud resources.
- Deploy the same tested application version across multiple cloud environments.
- Rolling updates can be used to release new versions with minimum downtime.

Service Discovery

- Kubernetes provides built-in service discovery using **Services and DNS**.
- Applications communicate using service names rather than fixed IP addresses.

- Service discovery automatically identifies available application instances.
- Health checks ensure that traffic is sent only to healthy services.

Scaling

- Use the **Horizontal Pod Autoscaler (HPA)** to increase or decrease containers according to workload.
- Use cluster autoscaling to add or remove computing nodes.
- Load balancing distributes requests among available containers.
- Resources can be increased automatically during high transaction periods.

Benefits of Multi-Cloud

- Provides **high availability** if one cloud provider experiences a failure.
- Reduces **vendor lock-in**.
- Improves disaster recovery capabilities.
- Provides flexibility to select suitable services from different providers.
- Workloads can be distributed based on performance and cost.

Risks

- Multi-cloud environments are more difficult to manage.
- Different providers may have different APIs and services.
- Data synchronization can become complex.
- Security policies must be maintained consistently.
- Data-transfer costs may increase.
- Monitoring and troubleshooting across multiple providers can be difficult.

Conclusion

Using **Kubernetes, containerization, CI/CD, service discovery, and auto-scaling** provides consistent and scalable application management across multiple cloud providers.

3. Backup and Disaster Recovery for an E-Learning Platform

The e-learning platform should use cloud-native backup, replication, and versioning to prevent permanent data loss and ensure quick recovery.

Cloud-Native Backup

- Configure automatic and scheduled backups for databases and important application data.
- Store backups separately from the primary production environment.
- Use cloud backup services to automate backup creation and retention.
- Encrypt backups to protect them from unauthorized access.
- Regularly verify that backups can be successfully restored.

Data Replication

- Replicate critical data to another availability zone or cloud region.
- Database replication maintains an additional copy of important data.
- Cross-region replication protects data from regional failures.
- Critical data can be replicated frequently to minimize data loss.

Versioning

- Enable **object versioning** for files stored in cloud storage.
- Previous versions of files can be restored after accidental deletion or modification.
- Use retention policies to prevent important versions from being deleted immediately.

RTO

Recovery Time Objective (RTO) is the maximum acceptable time required to restore the service after a failure.

- Use standby or replicated environments to reduce recovery time.
- Automate infrastructure deployment and recovery procedures.
- Prioritize critical services during recovery.
- Regular disaster recovery testing helps achieve the required RTO.

RPO

Recovery Point Objective (RPO) is the maximum acceptable amount of data that can be lost after a failure.

- Frequent backups provide a lower RPO.
- Continuous or near-real-time replication provides a very low RPO.

- Critical databases should be backed up or replicated more frequently than less important data.

Business Continuity Steps

1. Identify critical applications and data.
2. Define suitable RTO and RPO values.
3. Configure automatic backups.
4. Enable storage versioning.
5. Replicate critical data to another region.
6. Maintain a disaster recovery environment.
7. Automate recovery and failover procedures.
8. Regularly test backup restoration.
9. Monitor backup and replication failures.
10. Update the disaster recovery plan regularly.

Conclusion

A combination of **automated backups, data replication, versioning, and disaster recovery testing** minimizes data loss and helps the e-learning platform continue its services during failures.

4. Hybrid Cloud for a Government Agency

A hybrid cloud combines on-premises infrastructure with public cloud services. Sensitive applications can remain on-premises while non-sensitive workloads can use the public cloud.

Hybrid Cloud Implementation

- Store sensitive government data and compliance-critical applications on-premises.
- Deploy non-sensitive applications and workloads in the public cloud.
- Connect both environments using a secure **VPN or dedicated private connection**.
- Use firewalls to control communication between the environments.
- Apply network segmentation to isolate sensitive systems.
- Monitor traffic between on-premises and cloud environments.

Identity Management

- Use centralized **Identity and Access Management (IAM)**.
- Implement **Role-Based Access Control (RBAC)**.
- Provide users only the permissions required for their jobs.
- Use **Multi-Factor Authentication (MFA)** for sensitive and privileged accounts.
- Use Single Sign-On where appropriate.
- Maintain audit logs of authentication and user activities.

Encryption

- Encrypt sensitive information **at rest** using strong encryption.
- Encrypt information **in transit** using TLS and secure VPN connections.
- Protect encryption keys using a secure key-management service.
- Regularly rotate encryption keys.
- Restrict access to encryption keys according to user roles.

Challenges

- Managing on-premises and cloud resources together is complex.
- Maintaining consistent security policies can be difficult.
- Network connectivity between environments can become a bottleneck.
- Data migration and synchronization may be challenging.
- Compliance and data-residency requirements must be continuously monitored.
- Troubleshooting becomes more difficult because multiple environments are involved.
- Skilled cloud and security professionals are required.

Conclusion

A hybrid cloud provides **security and compliance for sensitive workloads** while providing the **scalability and flexibility of public cloud services** for non-sensitive workloads.

5. Multi-Region High Availability for a SaaS Provider

A SaaS provider can use a multi-region deployment so that the application remains available even if one cloud region becomes unavailable.

Multi-Region Deployment

- Deploy the application in at least two geographically separate cloud regions.
- Use multiple availability zones within each region.
- Deploy application servers in each region.
- Use a **global load balancer or DNS-based routing** to distribute users between regions.
- Direct users to a healthy and suitable region.
- Maintain sufficient computing capacity in the secondary region for failover.

Data Replication

- Replicate databases between the primary and secondary regions.
- Replicate important files and application data across regions.
- **Synchronous replication** provides stronger consistency but may introduce higher latency.
- **Asynchronous replication** provides better geographic performance but may allow a small amount of data loss.
- Monitor replication lag continuously.

Failover

- Continuously monitor the health of each region.
- Detect regional failures using automated health checks.
- Remove the failed region from traffic routing.
- Redirect users to the healthy region.
- Promote the secondary database if required.
- Restore the failed region after the problem is resolved.
- Synchronize data before returning the recovered region to normal operation.

Monitoring

Monitor:

- CPU and memory usage.
- Application response time.
- Request and error rates.

- Database health.
- Database replication lag.
- Network performance.
- Storage capacity.
- Server and load-balancer health.
- Overall application availability.

Alerting

- Configure automatic alerts when predefined thresholds are exceeded.
- Send critical alerts to the operations or cloud management team.
- Use centralized logs for troubleshooting.
- Use dashboards to monitor application health in real time.
- Use automated health checks for early failure detection.
- Regularly test failover procedures to ensure they work correctly.

Conclusion

A **multi-region deployment with data replication, global load balancing, automated failover, monitoring, and alerting** provides high availability and fault tolerance. It allows the SaaS application to continue operating even when one cloud region fails.